

CargoBit Transport Platform

Audit & Security Architecture

Platform-Erweiterungen für Produktionsbetrieb

Version 1.0 | April 2026

Executive Summary

Dieses Dokument definiert die produktionskritischen Architektur-Erweiterungen für die CargoBit Transport-Plattform. Die vier Kernbereiche - Audit-Logging, Anti-Fraud-Checks, Rate-Limiting und Data-Retention - bilden das Fundament für eine sichere, compliant und betriebsstabile Microservice-Landschaft. Jede Komponente ist so konzipiert, dass sie nahtlos in die bestehende Event-getriebene Architektur integriert wird, ohne die Performance der Kern-Workflows zu beeinträchtigen.

Die Implementierung folgt dem Prinzip "Security by Design" - Schutzmaßnahmen sind von Anfang an in jeden Service integriert, nicht als nachträgliche Add-ons. Alle Audit-Records werden in einem zentralen, append-only Store archiviert und sind über correlationId mit den ursprünglichen Business-Events verknüpft. Rate-Limits werden bereits auf API-Gateway-Ebene durchgesetzt, bevor Requests die Service-Grenzen erreichen.

Quick Reference

Komponente

1. Audit-Log Architektur

1.1 Zielsetzung

Jede geschäftskritische Entscheidung muss vollständig nachvollziehbar sein. Audit-Logs dienen nicht nur der Compliance (DSGVO, GoBD, steuerliche Aufbewahrungspflichten), sondern auch der Betriebsanalyse, Debugging und Betrugserkennung. Ein zentraler Audit-Store gewährleistet, dass Records nicht manipuliert oder gelöscht werden können (WORM - Write Once, Read Many).

Die Audit-Architektur folgt dem Prinzip der Dezentralität bei der Erfassung und Zentralität bei der Speicherung. Jeder Service schreibt seine Audit-Records asynchron in den zentralen Store, ohne dass der Business-Flow blockiert wird. Die correlationId ermöglicht es, alle Audit-Records einer Transaktion über Service-Grenzen hinweg zu verfolgen.

1.2 Audit-Punkte pro Service

Order-Service

Der Order-Service ist der Entry-Point für alle Transportaufträge. Jede Zustandsänderung wird auditiert, um die vollständige Auftrags-Historie nachvollziehbar zu machen.

Event

Pricing-Service

Der Pricing-Service berechnet Preise und validiert Gebote. Da Preis-Manipulation ein hohes Betrugspotenzial birgt, werden alle Preisberechnungen und Validierungen detailliert protokolliert - inklusive der verwendeten Konfigurations-Version.

Event

Matching-Service

Der Matching-Service trifft die geschäftskritische Entscheidung, welcher Carrier einen Auftrag erhält. Die vollständige Score-Berechnung und die Gewichtungsfaktoren werden für jedes Matching archiviert.

Event

Execution-Service

Der Execution-Service verwaltet den physischen Transportprozess. Status-Transitions und POD-Uploads sind rechtsrelevant und müssen lückenlos dokumentiert sein.

Event

Bidding-Service

Der Bidding-Service verwaltet alle Gebote eines Carriers für offene Aufträge. Gebotshistorie ist wichtig für Marktanalyse und Betrugserkennung.

Event

1.3 Audit-Record Schema (Generisch)

Das folgende JSON-Schema definiert die Struktur jedes Audit-Records. Die Felder sind so gew□hlt, dass sie sowohl f□r Compliance-Anforderungen als auch f□r operative Analysen ausreichend Information tragen, ohne sensible Daten unn□tig zu exponieren.

```
{ "id": "audit_01HXYZ123456789", "timestamp": "2026-04-17T20:00:00.000Z", "actorType": "user | system | service", "actorId": "ship_1 | car_2 | svc_pricing", "service": "pricing-service", "action": "BID_VALIDATED", "entityType": "bid", "entityId": "bid_999", "payloadBefore": null, "payloadAfter": { "valid": true, "priceScore": 0.72, "validationResult": "APPROVED" }, "correlationId": "trace_abc123", "metadata": { "ipAddress": "192.168.1.100", "userAgent": "CargoBit-Carrier-App/2.1", "sessionId": "sess_xyz" } }
```

Schema-Felder erklärt:

[illegible]

1.4 Audit-Store Architektur

Der zentrale Audit-Store wird als separater Service mit eigener Datenbank betrieben. Er empfängt Audit-Records asynchron über den Event-Bus und schreibt sie in eine append-only, WORM-fähige Datenbank (z.B. Amazon S3 Object Lock, Azure Immutable Storage, oder eine spezialisierte Audit-DB wie AuditDB).

Wichtige Design-Entscheidungen:

☐ Append-Only: Records können nur hinzugefügt, niemals geändert oder gelöscht werden ☐ WORM-Fähigkeit: Write-Once-Read-Many Schutz gegen Manipulation ☐
Zeitindex: Automatischer Index auf timestamp für effiziente Zeitbereichs-Abfragen ☐
CorrelationIndex: Index auf correlationId für Cross-Service-Tracing ☐ EntityIndex: Index auf (entityType, entityId) für Entities-Historie ☐ Separation of Concerns: Audit-DB ist getrennt von operativen DBs ☐ Async Write: Services blockieren nicht auf Audit-Writes (Event-Bus) ☐ Retention: 7 - 10 Jahre je nach Compliance-Anforderungen

2. Anti-Fraud-Checks in Pricing & Matching

2.1 Zielsetzung

Betrugsprävention ist in einer B2B-Transportplattform kritisch. Die Hauptbedrohungen sind Preismanipulation (Dumping), Collusion (Absprachen zwischen Carriern), und ungewöhnliches Bietverhalten. Anti-Fraud-Checks werden auf drei Ebenen durchgeführt: Pricing-Service (Preis-Validierung), Bidding-Service (Gebots-Muster), und Matching-Service (Ergebnis-Analyse).

2.2 Pricing-Service: Bid-Validation

Jedes Gebot wird validiert bevor es in den Matching-Prozess eingeht. Die Validierung prüft gegen definierte Preisgrenzen und Marktbedingungen.

Check

2.3 Bidding-Service: Rate-Limits & Pattern Detection

Der Bidding-Service überwacht das Gebotsverhalten pro Carrier und pro Order. Auffällige Muster werden automatisch erkannt und können zu temporären Sperren oder manuellen Reviews führen.

Check

2.4 Matching-Service: Collusion Detection

Der Matching-Service analysiert Matching-Ergebnisse auf Muster, die auf Absprachen hindeuten. Diese Checks laufen asynchron nach dem Matching und flaggen verdächtige Konstellationen.

Pattern

2.5 Risk-Service: Carrier Fraud Score

Der Risk-Service berechnet einen kontinuierlichen Fraud-Score für jeden Carrier basierend auf historischem Verhalten. Der Score beeinflusst das Matching-Gewichtung und kann bei kritischen Werten zu automatischen Sperren führen.

Faktor

Fraud-Score Thresholds:

☐ Score < 0.3: Normal - keine Einschränkungen ☐ Score 0.3 - 0.6: Elevated - Matching-Penalty (-20% Score), manuelles Monitoring ☐ Score 0.6 - 0.8: High - Matching-Penalty (-50% Score), Alert an Risk-Team ☐ Score > 0.8: Critical - Automatische Sperre, manuelles Review erforderlich

3. Rate-Limiting Strategie

3.1 Zielsetzung

Rate-Limiting schützt die Plattform vor Überlastung, Missbrauch und DoS-Angriffen. Es wird auf zwei Ebenen implementiert: API-Gateway (globale Limits) und Service-Level (granulare Limits pro Business-Logik). Die Limits sind so gewählt, dass legitime Nutzer nicht beeinträchtigt werden, während abusive Muster effektiv blockiert werden.

3.2 API-Gateway Rate-Limits

Das API-Gateway (Kong, APIM, oder NGINX) setzt die ersten Rate-Limits durch, bevor Requests die internen Services erreichen. Implementierung erfolgt typischerweise via Redis-basiertem Token-Bucket oder Leaky-Bucket Algorithmus.

Endpoint

3.3 Implementierung: Token-Bucket Algorithmus

Der Token-Bucket Algorithmus erlaubt Burst-Traffic bis zu einem definierten Maximum, während die durchschnittliche Rate langfristig begrenzt wird. Redis speichert den Bucket-Status mit atomaren Operationen.

```
// Redis-basierter Token-Bucket function checkRateLimit(key: string, maxTokens: int, refillRate: int): boolean {
const now = Date.now() const bucket = redis.hgetall(key) if (!bucket) { // Neuer Bucket redis.hmset(key, {tokens:
maxTokens - 1, lastRefill: now}) redis.expire(key, 3600) // 1h TTL return true } // Tokens nachfüllen const elapsed
= now - parseInt(bucket.lastRefill) const refillTokens = Math.floor(elapsed / 1000 * refillRate) const
currentTokens = Math.min(maxTokens, parseInt(bucket.tokens) + refillTokens) if (currentTokens < 1) { return
false // Rate-Limit exceeded } redis.hmset(key, {tokens: currentTokens - 1, lastRefill: now}) return true }
```

3.4 Rate-Limit Response Headers

Jede API-Antwort enthlt Header, die den aktuellen Rate-Limit-Status kommunizieren. Dies ermglicht Clients, sich gracefully zu verhalten und Backoff-Strategien zu implementieren.

Header

3.5 429 Response Format

```
{ "error": { "code": "RATE_LIMIT_EXCEEDED", "message": "Rate limit exceeded. Please retry after 23 seconds.", "details": { "limit": 120, "window": "60s", "retryAfter": 23 } } }
```

4. Data-Retention Policies

4.1 Prinzipien

Data-Retention folgt dem Prinzip "So viel wie nötig, so wenig wie möglich - aber audit-fähig". Jeder Service definiert seine Retention-Policies basierend auf rechtlichen Anforderungen (Steuer, Compliance), betrieblichen Notwendigkeiten und Datenschutz (DSGVO). Operative Daten werden nach Ablauf der Retention automatisch archiviert oder gelöscht.

Wichtige Grundsätze:

- ☐ Rechtliche Mindestaufbewahrung wird immer eingehalten
- ☐ DSGVO-Rechte (Löschung auf Antrag) sind implementiert, mit Ausnahme für Audit-Daten
- ☐ Archivierung erfolgt auf Cold Storage (S3 Glacier, Azure Archive)
- ☐ Purge-Jobs laufen täglich und löschen/archivieren abgelaufene Daten
- ☐ Audit-Logs haben immer die längste Retention (7 - 10 Jahre)
- ☐ POD-Dokumente werden separat mit langer Retention verwaltet

4.2 Retention pro Service und Datentyp

Service

4.3 Archivierungs-Pipeline

Daten werden nicht sofort gelöscht, sondern durchlaufen eine Archivierungs-Pipeline. Dies ermöglicht Wiederherstellung bei Bedarf und reduziert Storage-Kosten durch Cold-Storage-Migration.

ARCHIVIERUNGS-PIPELINE									
Cold Storage → Delete			(PostgreSQL) (S3 Standard) (S3 Glacier)		Hot Storage → Warm Storage →				
• Vollzugriff • Seltener Zugriff • Audit-only			• Aktive Daten • 1–3 Jahre • 3–7 Jahre		• Keine Kosten • Niedrige Kosten • Minimale Kosten				
Übergänge:			Hot → Warm: Nach 1 Jahr (automatisch via Job)		Warm → Cold: Nach 3 Jahren (automatisch via Job)				
			Cold → Delete: Nach Retention-Ende (manuell freigegeben)						

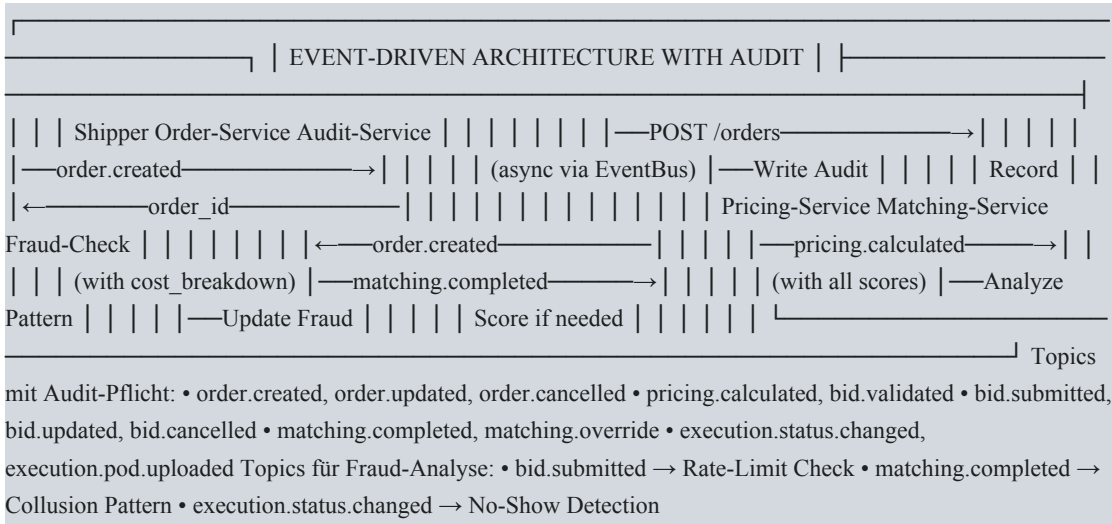
4.4 Purge-Job Konfiguration

[illegible]

5. Service-übergreifende Integration

5.1 Event-Flow mit Audit & Fraud

Die folgende Darstellung zeigt, wie Audit-Logging und Fraud-Checks in den bestehenden Event-Flow integriert werden. Jedes geschäftsrelevante Event wird parallel auditiert, und Fraud-Checks laufen als asynchrone Analysen.



5.2 Service-Responsibilities Matrix

Service

5.3 Implementierungs-Prioritäten

Priorität

Dokument-Hinweis: Dieses Architektur-Dokument ist die Basis für die Implementierung der Produktions-Erweiterungen. Alle Spezifikationen sollten vor der Implementierung mit dem Legal- und Compliance-Team abgestimmt werden, um regionale Anforderungen (DSGVO, GoBD, etc.) vollständig zu berücksichtigen.